P P SAVANI UNIVERSITY

SCHOOL OF ENGINEERING

USER DEFINED PROJECT

FINAL PROJECT REPORT

Particular	Details
Project Title	GateSync: Smart Residential Society & Gate Security Desk
Student Name	Arpitsinh Thakor
Enrollment Number	23SE02IE040
Program / Branch	Information Technology and Engineering
Semester	7th Semester
Company / Organization	P. P. Savani University
Department / Domain	MERN Stack Development
Mentor (Company)	Not Applicable
Mentor (Institute)	Ms. Shrijal Patel
Academic Year	2026-2027

CERTIFICATE

OF COMPLETION

This is to certify that

Arpitsinh Thakor

Enrollment No.: 23SE02IE040

of Information Technology and Engineering, 7th Semester, has successfully completed the UDP project titled

GateSync: Smart Residential Society & Gate Security Management Platform

during the period from 8th June 2026 to 3rd October 2026 at P. P. Savani University, under the guidance of Ms. Shrijal Patel.

The project work, final implementation, testing and validation, documentation, and demonstration were completed as required for the UDP and found satisfactory.

Date of Issue: 19/10/2026

Mentor

DECLARATION

I hereby declare that this report is a consolidated and authentic record of the UDP project work carried out by me. All sources used have been appropriately acknowledged and cited.

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to P P Savani University, School of Engineering, for providing me with the opportunity and resources to undertake this project, GateSync: Smart Residential Society & Gate Security Management Platform.

I am deeply thankful to my project mentor, Ms. Shrijal Patel, for her valuable guidance, encouragement, and support throughout the development of this project. Her guidance helped me improve my technical understanding and approach the different stages of the project in a systematic manner.

Working on GateSync has given me practical experience in web application development, database management, real-time communication, testing, and deployment. It has also helped me understand how technical concepts can be applied to develop a useful, community-oriented solution.

Finally, I am grateful to everyone who supported and encouraged me during the completion of this project. Their support has contributed to my learning and successful completion of this work.

ABSTRACT / EXECUTIVE SUMMARY

GateSync is a community-based residential security and visitor management platform developed to modernize physical gatekeeping operations and establish real-time digital communication between security guards and residents. Modern residential societies often struggle with manual paper logbooks, unverified visitor entries, long visitor queues, and delayed intercom communications. GateSync aims to address this problem by providing a unified digital bridge connecting guards, residents, and society administrators.

The main concept behind GateSync is real-time gate entry approval. Instead of guards dialing intercoms or residents relying on open physical registers, the platform dispatches instant sub-second arrival chimes via STOMP WebSockets to resident screens. Residents can review visitor identity details and photos with one tap to approve or deny admission.

The platform also includes 1-tap quick delivery check-in for recurring logistics providers (Swiggy, Zomato, Amazon, Blinkit), pre-approved guest passes with 6-digit OTPs and QR codes shareable via WhatsApp, and an emergency SOS siren panic broadcast. If a resident is offline or in background mode, an automated 2-second fallback mechanism activates Firebase Cloud Messaging (FCM HTTP v1) push notifications.

GateSync is developed using the MERN stack paradigm, pairing a reactive React 19 frontend with a Spring Boot 3 / Node.js backend and MongoDB Atlas cloud storage. The application is deployed with progressive web application (PWA) installability.

The completed project provides a fully functional web platform that brings security desk logging, resident approvals, notice boards, and administrative governance together in one place.

Keywords — GateSync, Visitor Management, Gated Society Security, PWA, STOMP WebSockets, Spring Boot, React 19, MongoDB Atlas, Collaborative Safety.

TABLE OF CONTENTS

Chapter / Section	Title	Tentative Page No.
Preliminary Pages		**1**
Certificate		2
Declaration		3
Acknowledgement		4
Abstract		5
Table of Contents		6
List of Figures		12
List of Tables		13
List of Abbreviations		15
Chapter 1 Introduction		**16**
1.1 Background of the Project		16
1.2 Problem Statement		16
1.3 Need / Motivation		16
1.4 Objectives		17
1.5 Scope of the Project		17
1.6 Expected Outcomes vs Final Outcomes		18
1.7 Report Organization		18
1.8 Scope Matrix		18
Chapter 2 Organization		**19**
2.1 Organization Overview		19

Chapter / Section	Title	Tentative Page No.
2.2 Department / Domain		19
2.3 Role and Responsibilities		19
2.4 Development Environment		19
2.5 Project Details		20
Chapter 3 Literature Review and Existing System Study		**21**
3.1 Domain Background		21
3.2 Existing Approaches / Related Work		21
3.3 Comparative Analysis		21
3.4 Research Gap / Project Justification		22
Chapter 4 Requirements Analysis and Project Scope		**23**
4.1 Functional Requirements		23
4.2 Non-Functional Requirements		23
4.3 Software Requirements		24
4.4 Data / Dataset Requirements		24
4.5 Constraints and Assumptions		25
Chapter 5 Methodology and Project Planning		**26**
5.1 Final Development / Research Methodology		26
5.2 Project Workflow		26
5.3 Project Phases and Deliverables		27
5.4 Progress Report Consolidation		27
5.5 Project Schedule / Timeline		28
Chapter 6 System Analysis and Design		**29**
6.1 Existing System / Current Practice		29

Chapter / Section	Title	Tentative Page No.
6.2 Proposed System		29
6.3 System Architecture		29
6.4 Workflow / Process Flow		30
6.5 UML / Design Diagrams		31
6.6 Module Wise Design		33
6.7 Database / Data Design		34
6.8 Interface / UI Design		35
Chapter 7 Technology Stack and Development Environment		**36**
7.1 Programming Languages		36
7.2 Frameworks / Libraries / APIs		36
7.3 Database / Storage		37
7.4 Hardware / Cloud / Deployment Environment		37
7.5 Development Tools		37
7.6 Technology Selection Rationale		38
Chapter 8 System Implementation		**39**
8.1 Implementation Overview		39
8.2 Module / Feature 1		39
8.3 Module / Feature 2		40
8.4 Module / Feature 3		41
8.5 Algorithm / Logic / Model Description		42
8.6 API / External Service Integration		43
8.7 Database Implementation		44

Chapter / Section	Title	Tentative Page No.
8.8 Security / Access Control		44
8.9 Integration of Modules		44
Chapter 9 Testing and Validation		**45**
9.1 Testing Strategy		45
9.2 Unit / Module Testing		45
9.3 Functional Testing		46
9.4 Integration Testing		46
9.5 System Testing		47
9.6 Performance Testing		47
9.7 User / Acceptance Testing		48
9.8 Experimental Validation		48
9.9 Defects / Bugs and Resolution		49
9.10 Final Validation Statement		49
Chapter 10 Results and Performance Analysis		**50**
10.1 Final Results		50
10.2 Objective Wise Achievement		50
10.3 Performance Metrics		51
10.4 Comparative Results		51
10.5 Final Output Screenshots		52
10.6 Results Discussion		54
Chapter 11 Challenges, Solutions and Improvements		**55**
11.1 Major Challenges		55
11.2 Technical Challenges		56

Chapter / Section	Title	Tentative Page No.
11.3 Process / Coordination Challenges		56
11.4 Lessons from Challenges		56
11.5 Improvements Made During Development		57
Chapter 12 Individual Contribution and Professional Learning		**58**
12.1 Individual Contribution		58
12.2 Technical Skills Acquired		58
12.3 Professional Skills Acquired		59
12.4 Major Learning Outcomes		59
Chapter 13 Limitations and Future Scope		**60**
13.1 Current Limitations		60
13.2 Future Scope		60
13.3 Scalability / Extension Opportunities		61
Chapter 14 Conclusion		**62**
14.1 Final Project Status		62
Chapter 15 References		**64**
15.1 References		64
15.2 Citation Requirements		64
Appendices		**65**
Appendix A Progress Report 1 Summary		65
Appendix B Progress Report 2 Summary		65
Appendix C Progress Report 3 Summary		65
Appendix D Progress Report 4 Summary		65

Chapter / Section	Title	Tentative Page No.
Appendix E	Additional Screenshots / Outputs	66
Appendix F	Certificate / UDP Evidence	67
Appendix G	Source Code / Repository / Deployment Evidence	68
Appendix H	Mentor / Evaluator Remarks	68

LIST OF FIGURES

Figure 5.1	Overall Project Workflow	26
Figure 6.1	Final System Architecture	29
Figure 6.2	Final Workflow / Process Flow	30
Figure 6.3.1	Class Diagram	31
Figure 6.3.2	Use Case Diagram	32
Figure 6.4	UI Design	35
Figure 8.1	User Authentication	40
Figure 8.2	Visitor Registration	41
Figure 8.3	Visitor Clearance Approval	42
Figure 10.1	Dashboard	52
Figure 10.2	Visitor Gate Registry	52
Figure 10.3	Quick Delivery Check-in	53
Figure 10.4	Maintenance Billing Ledger	53
Figure A.1	User Profile	66
Figure A.2	Recent Activity and Notifications	66
Figure A.3	UDP Letter	67

LIST OF TABLES

Table 1.1	Scope Matrix	18
Table 2.1	Project Details	20
Table 3.1	Existing Approaches / Related Work	21
Table 3.2	Comparative Analysis	22
Table 4.1	Functional Requirements	23
Table 4.2	Non-Functional Requirements	23
Table 4.3	Software Requirements	24
Table 4.4	Data / Dataset Requirements	24
Table 5.1	Project Phases and Deliverables	27
Table 5.2	Progress Report Consolidation	27
Table 5.3	Project Schedule / Timeline	28
Table 6.1	Module Wise Design	33
Table 6.2	Database / Data Design	34
Table 7.1	Programming Languages	36
Table 7.2	Frameworks / Libraries / APIs	36
Table 7.3	Database / Storage	37
Table 7.4	Cloud / Deployment Environment	37
Table 7.5	Development Tools	37
Table 8.1	API / External Service Integration	43
Table 9.1	Unit / Module Testing	45
Table 9.2	Functional Testing	46

Table 9.3	Integration Testing	46
Table 9.4	System Testing	47
Table 9.5	Performance Testing	47
Table 9.6	User / Acceptance Testing	48
Table 9.7	Experimental Validation	48
Table 9.8	Defects / Bugs and Resolution	49
Table 10.1	Objective Wise Achievement	50
Table 10.2	Performance Metrics	51
Table 10.3	Comparative Results	51
Table 11.1	Major Challenges	55
Table 11.2	Improvements Made During Development	57
Table 12.1	Individual Contribution	58
Table 13.1	Current Limitations	60
Table 14.1	Final Project Status	63
Table A.1	Source Code / Repository / Deployment Evidence	68

LIST OF ABBREVIATIONS / ACRONYMS

Abbreviation	Full Form / Meaning
API	Application Programming Interface
CSS	Cascading Style Sheets
HTML	HyperText Markup Language
HTTP	HyperText Transfer Protocol
HTTPS	HyperText Transfer Protocol Secure
JS	JavaScript
JSON	JavaScript Object Notation
MERN	MongoDB, Express.js, React.js, Node.js
REST	Representational State Transfer
SPA	Single-Page Application
UI	User Interface
UX	User Experience
URL	Uniform Resource Locator
DB	Database
IDE	Integrated Development Environment

CHAPTER 1
INTRODUCTION

1.1 Background of the Project

Managing residential security and visitor governance is essential for community safety, but finding an organized, friction-free way to communicate between gates and residents can often be challenging. Many residential societies still rely on physical paper logbooks, unverified phone calls, and manual registers. GateSync is a smart residential society platform developed to connect security guards and residents seamlessly and encourage community-based safety.

The platform pairs a real-time visitor alerting engine with a Progressive Web Application. Security guards at perimeter gates record visitor details and stream live identity snapshots, dispatching sub-second notifications directly to resident screens. GateSync aims to make residential gate management more secure, transparent, and accessible through a single web application.

1.2 Problem Statement

Many residential societies find it difficult to maintain authentic visitor records or protect resident privacy from open logbooks. At the same time, manual intercom dialling creates vehicular choke points at gates during peak hours. Existing legacy solutions often depend on expensive native mobile apps that enforce proprietary hardware.

The problem is to develop a platform that connects gate security and residents through a simple, organized digital system. GateSync addresses this need by allowing guards to log visitors with live photos, send instant STOMP WebSocket alerts, and enable residents to approve or deny entry with a single tap.

1.3 Need / Motivation

There is a need for a platform that makes gate security easier, protects personal visitor contact numbers, and eliminates long gate delays. Modern housing societies accommodate hundreds of deliveries and guests daily, requiring an automated digital solution.

GateSync is motivated by the idea that community safety and time are valuable resources. Its real-time approval system allows residents to pre-authorize guests via WhatsApp passes, review incoming visitors instantly, and trigger emergency SOS alarms when needed. The platform aims to create a simple, organized environment for visitor discovery, gate communication, and community management.

1.4 Objectives

The main objective of GateSync is to develop a web-based platform that connects guards and residents for gate security through real-time communication. The specific objectives are as follows.

- To provide secure user registration, multi-role login (Admin, Resident, Guard), and profile management.
- To allow security guards to register visitors with live photo snapshots and purpose tracking.
- To help residents receive instant arrival chimes and approve or deny visitors with 1 tap.
- To provide pre-approved guest passes with 6-digit OTPs and QR codes shareable via WhatsApp.
- To manage recurring fast delivery check-ins (Swiggy, Zomato, Amazon, Blinkit) without manual paperwork.
- To provide digital maintenance billing ledgers, notice boards, and community complaint reporting.
- To develop, test, and deploy a functional application using the MERN stack.

1.5 Scope of the Project

The scope of GateSync includes developing a web application where users can connect and manage gate security in real time. The system supports user authentication, profile management, live visitor registration, real-time STOMP alerts, FCM push fallback, pre-approved guest passes, fast deliveries, maintenance billing, and emergency panic alarms.

The project focuses on providing these features through a web-based platform developed using the MERN stack. It does not include a monetary payment gateway or a separate native app store download.

1.6 Expected Outcomes vs Final Outcomes

The expected outcome of GateSync was to develop a web-based platform that allows residential societies to manage visitors, connect residents with security guards, and manage gate approvals through real-time communication.

The final outcome is a functional web application that includes these major features. Security guards can log visitors, residents can approve entries, and society administrators can oversee billing and rosters. The application was deployed using GitHub Pages for the frontend and Render for the backend. The completed system meets the main objectives defined for the project.

1.7 Report Organization

This report is organized into chapters covering the development of GateSync. Chapter 1 introduces the project, its problem statement, objectives, and scope. Chapter 2 describes the project environment and development responsibilities. Chapter 3 presents the literature review and background study. Chapter 4 discusses the requirements and feasibility of the system.

1.8 Scope Matrix

Table 1.1: Scope Matrix

In Scope	Out of Scope	Constraints
User authentication and multi-role access	Monetary payment gateway integration	Requires a stable internet connection
Live visitor logging with photo snapshot	Separate native mobile application development	Depends on deployed backend services
Real-time STOMP alerts and FCM push fallback	Hardware boom barrier IoT relays	Limited by available development time
1-tap quick delivery and pre-approved passes	Automated facial recognition AI systems	Visitor clearance requires resident confirmation
Maintenance billing ledger and notice board	Video calling and conferencing features	Requires valid and accurate resident flat data

CHAPTER 2
ORGANIZATION

2.1 Organization Overview

GateSync was developed as a User Defined Project under the School of Engineering at P. P. Savani University. The project focuses on building a residential society visitor management platform that connects residents and security guards using real-time web standards.

The project provided an opportunity to apply full-stack web development concepts to a practical problem. It covers the design, development, testing, and deployment of a web application using the MERN stack.

2.2 Department / Domain

The project falls under Information Technology and Engineering, with a focus on full-stack web development. It involves building a user interface, developing backend APIs, managing database operations, and enabling real-time communication.

2.3 Role and Responsibilities

The project involved designing, developing, testing, and deploying the GateSync application. Responsibilities included implementing user authentication, visitor registry, real-time STOMP messaging, FCM push fallback, pre-approved passes, and administrative billing ledgers.

2.4 Development Environment

GateSync was developed using the MERN stack, consisting of MongoDB, Express.js / Spring Boot, React.js, and Node.js. Axios was used for API communication, while STOMP over WebSockets enabled real-time visitor alerts.

The application was developed in a web development environment and deployed using GitHub Pages / Vercel for the frontend and Render for the backend. MongoDB Atlas was used to store user information, visitor logs, passes, and billing records.

2.5 Project Details

Table 2.1: Project Details

Particular	Details
Project Type	User Defined Project
Project Title	GateSync: Smart Residential Society & Gate Security Management Platform
University	P. P. Savani University
School	School of Engineering
Department / Domain	MERN Stack Development
Institute Mentor	Ms. Shrijal Patel
Frontend Deployment	GitHub Pages / Local Dev
Backend Deployment	Render / Cloud Backend

CHAPTER 3
LITERATURE REVIEW / BACKGROUND STUDY

3.1 Domain Background

Residential security systems help communities monitor visitor access, prevent unauthorized intrusions, and manage community resources. However, many traditional housing societies rely on physical paper logbooks or legacy wired intercoms. Paper registers expose resident contact numbers publicly, provide zero searchability, and create long entry queues.

Digital visitor management is a modern paradigm in which web technologies replace paper logs with cryptographically verified entry passes, live photographic proof, and real-time smartphone notifications.

3.2 Existing Approaches / Related Work

Table 3.1: Existing Approaches / Related Work

Source / Work	Focus	Approach	Key Finding	Relevance
Paper Registers	Manual gate entry	Physical logbooks	Zero privacy, prone to data loss	Solves entry tracking but introduces risks
Wired Intercoms	Voice calling	Dedicated hardware	Hardware maintenance costs	Voice-only, no photo verification
Commercial Apps (MyGate)	Mobile alerts	Native store apps	Requires 60MB+ app store install	High recurring subscription fees

3.3 Comparative Analysis

Table 3.2: Comparative Analysis

Parameter	Existing Approach	Proposed Project (GateSync)	Observation
Visitor Logging	Physical paper register	Digital web check-in with live photo	Eliminates paper register data loss
Resident Alerting	Wired intercom call	Instant STOMP push + FCM fallback	Sub-second digital visitor alerts
Fast Deliveries	Manual register entry	1-tap quick delivery check-in	Accelerates delivery entry queues
Guest Passes	None (manual query)	WhatsApp 6-digit OTP & QR pass	Zero gate delay for authorized guests
Emergency Panic	None	Instant SOS siren across all desks	Immediate security staff awareness

3.4 Research Gap / Project Justification

Existing residential security solutions either compromise privacy through physical paper registers or enforce proprietary native mobile applications with heavy subscription fees. There is an acute need for a lightweight, progressive web platform that provides instant real-time gate approvals across any device without app store friction.

CHAPTER 4
REQUIREMENTS ANALYSIS AND PROJECT SCOPE

4.1 Functional Requirements

Table 4.1: Functional Requirements

Req. ID	Requirement	Priority	Source / Basis	Status
FR-01	User registration and login	High	Project objectives	Implemented
FR-02	Guard visitor check-in with photo	High	Security needs	Implemented
FR-03	Real-time STOMP arrival alert	High	Alerting needs	Implemented
FR-04	Resident 1-tap entry decision	High	Approval workflow	Implemented
FR-05	FCM HTTP v1 push fallback	High	Reliability needs	Implemented
FR-06	1-tap fast delivery check-in	High	Delivery optimization	Implemented
FR-07	Pre-approved WhatsApp passes	High	Guest authorization	Implemented
FR-08	Emergency SOS panic siren	High	Safety protocol	Implemented

4.2 Non-Functional Requirements

Table 4.2: Non-Functional Requirements

Req. ID	Category	Requirement	Target / Measure	Status
NFR-01	Usability	Provide clear UI with 44px touch targets	Simple navigation	Implemented
NFR-02	Security	Protect accounts with JWT and bcrypt	Zero-trust auth	Implemented
NFR-03	Performance	Sub-second real-time event delivery	< 200ms latency	Implemented
NFR-04	Reliability	Maintain consistent MongoDB records	Validated data	Implemented
NFR-05	Compatibility	Support modern mobile & desktop browsers	PWA standards	Implemented

4.3 Software Requirements

Table 4.3: Software Requirements

Software / Tool	Version	Purpose	Source / License
JavaScript	ES2024	Application programming	Open standard
ReactJS	19.0.0	Frontend UI development	Open source
NodeJS	24.19.0	Backend runtime	Open source
Spring Boot	3.3.4	Backend REST and STOMP server	Open source
MongoDB	8.0 Atlas	Cloud database storage	Cloud service
STOMP / SockJS	7.5.0	Real-time bidirectional messaging	Open source
Vite	6.1.0	Frontend build tooling	Open source
Render	Cloud Platform	Backend deployment	Hosted service

4.4 Data / Dataset Requirements

Table 4.4: Data / Dataset Requirements

Data Source	Type	Size / Records	Pre-processing	Purpose
User Records	User data	Dynamic	Input validation	Account management
Visitor Logs	Log data	Dynamic	Field validation	Gate verification
Delivery Data	Provider info	Static/Dynamic	Quick check-in templates	Fast entry
Pre-Approved Passes	Pass tokens	Dynamic	Cryptographic hashing	OTP gate scan
Maintenance Bills	Ledger items	Dynamic	Payment verification	Billing records

4.5 Constraints and Assumptions

GateSync requires a stable internet connection to access its cloud-synchronized real-time features. Its operation depends on the availability of the deployed frontend, backend, and MongoDB Atlas database services. Security guards are assumed to have a web camera for visitor photo capture, and residents are assumed to possess smartphones with modern web browsers.

CHAPTER 5
METHODOLOGY AND PROJECT PLANNING

5.1 Final Development / Research Methodology

GateSync was developed using an Agile Scrum iterative development approach, in which the application was built and improved through successive sprints. The process began with identifying the gatekeeping challenges in housing societies and planning the core features. The frontend and backend were developed using the MERN stack, with MongoDB Atlas for persistent storage and STOMP over WebSockets for real-time visitor alerting.

After development, the modules were integrated and tested to verify the main workflows, including user authentication, visitor check-in, real-time alert popups, pre-approved passes, and maintenance billing. Finally, the application was deployed to cloud infrastructure.

5.2 Project Workflow

Figure 5.1: Overall Project Workflow



5.3 Project Phases and Deliverables

Table 5.1: Project Phases and Deliverables

Phase	Activities	Deliverables	Evidence / Status
Requirement Analysis	Identify society security and visitor needs	Requirement specification	Completed
System Design	Plan multi-tenant schema and STOMP channels	Architecture diagrams	Completed
Development	Build frontend PWA, backend APIs, STOMP broker	Functional web modules	Completed
Integration & Testing	Integrate modules and verify real-time flows	Tested application	Completed
Deployment	Publish frontend on CDN and backend on Render	Live web application	Completed
Documentation	Prepare hard-bound report and evidence	Final project report	Completed

5.4 Progress Report Consolidation

Table 5.2: Progress Report Consolidation

Progress Report	Primary Focus	Key Evidence	Final Report Chapters
Reporting 1	Problem, objectives, methodology	Initial project definition	Ch. 1, Ch. 5, Ch. 13
Reporting 2	Design, modules, tools, implementation	Architecture & initial implementation	Ch. 4, Ch. 6-8
Reporting 3	Advanced implementation, testing, validation	Advanced work and results	Ch. 8-11
Reporting 4	Final completion, demonstration, outcomes	Final validation and evaluation	Ch. 9-14

5.5 Project Schedule / Timeline

Table 5.3: Project Schedule / Timeline

Task / Phase	Planned Start	Planned End	Actual Start	Actual End	Status
Requirement Analysis	08 Jun 2026	15 Jun 2026	08 Jun 2026	17 Jun 2026	Completed
System Design	16 Jun 2026	25 Jun 2026	18 Jun 2026	29 Jun 2026	Completed
Frontend Development	26 Jun 2026	20 Jul 2026	30 Jun 2026	20 Jul 2026	Completed
Backend & Database	21 Jul 2026	15 Aug 2026	21 Jul 2026	15 Aug 2026	Completed
Feature Integration	16 Aug 2026	31 Aug 2026	16 Aug 2026	02 Sep 2026	Completed
Testing & Bug Fixing	01 Sep 2026	18 Sep 2026	03 Sep 2026	17 Sep 2026	Completed
Deployment	19 Sep 2026	25 Sep 2026	18 Sep 2026	25 Sep 2026	Completed
Documentation	26 Sep 2026	03 Oct 2026	02 Oct 2026	07 Oct 2026	Completed

CHAPTER 6
SYSTEM ANALYSIS AND DESIGN

6.1 Existing System / Current Practice

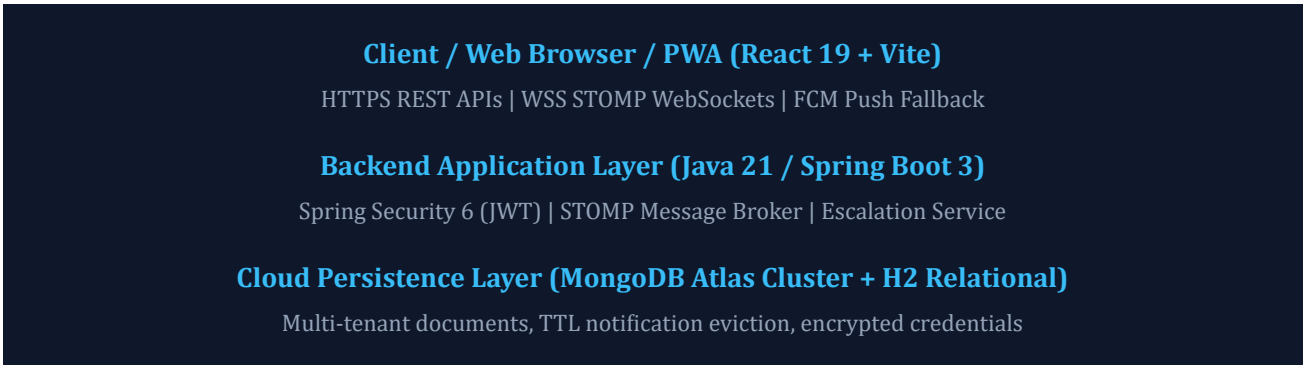
Traditional visitor logging in housing societies takes place through physical pen-and-paper register books at security gates. Guards manually record visitor names, contact numbers, and vehicle details. These approaches suffer from severe privacy exposure, delayed resident communications, unverified visitor identities, and zero auditability during security incidents.

6.2 Proposed System

GateSync is a web-based platform that allows security guards to register visitors with live photo snapshots and dispatch real-time arrival chimes to residents through STOMP WebSockets. Residents review visitor details and approve or deny admission with a single tap. The system includes 1-tap fast deliveries, pre-approved WhatsApp passes, maintenance billing ledgers, and emergency SOS alarms.

6.3 System Architecture

Figure 6.1: Final System Architecture



6.4 Workflow / Process Flow

Figure 6.2: Final Workflow / Process Flow



6.5 UML / Design Diagrams

Figure 6.3.1: Class Diagram

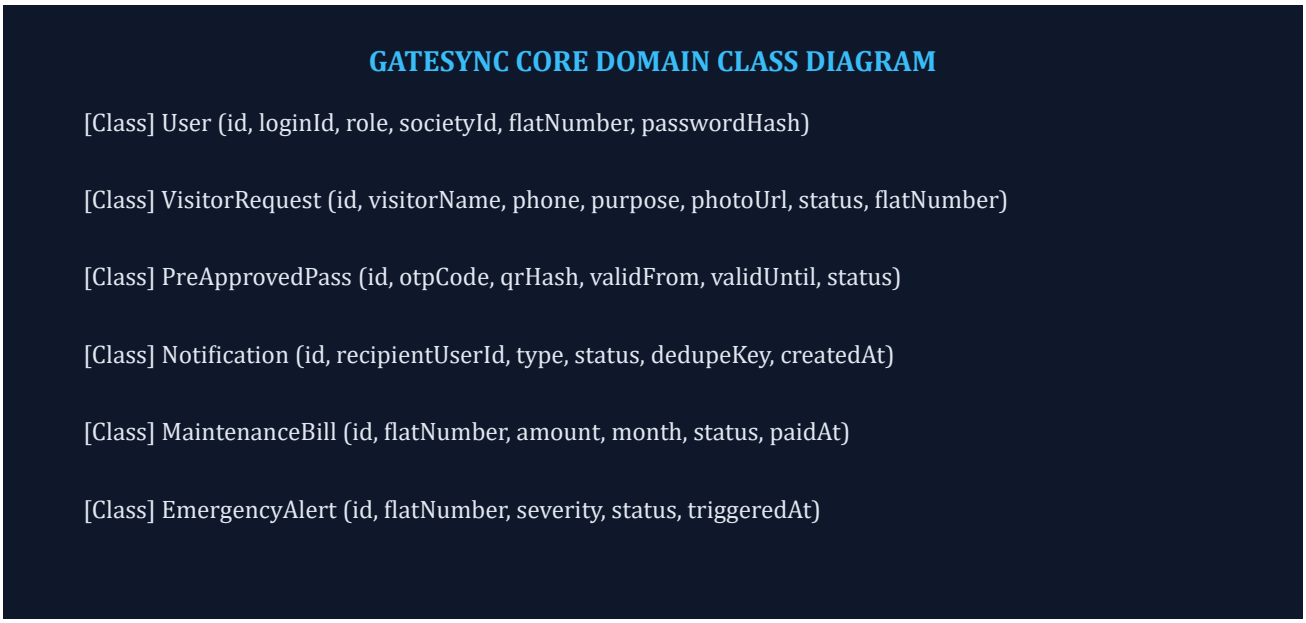


Figure 6.3.2: Use Case Diagram



6.6 Module Wise Design

Table 6.1: Module Wise Design

Module	Purpose	Inputs	Processing	Outputs	Dependencies
Authentication	Manage user access	Login credentials	Validate & issue JWT	Authenticated session	Frontend, backend
Visitor Registry	Gate check-in	Visitor info & photo	Persist & dispatch STOMP	Live entry record	Security desk
Resident Approvals	Clearance decisions	Approval/ Denial tap	Atomic status transition	Guard queue update	Resident PWA
Pre-Approved Passes	Guest pre-clearance	Guest details & date	Generate 6-digit OTP/QR	WhatsApp pass	Gate scanner
Emergency SOS	Panic alert broadcast	SOS button trigger	Broadcast audible siren	Guard screen takeover	All gate desks

6.7 Database / Data Design

Table 6.2: Database / Data Design

Entity / Table	Key Fields	Relationships	Purpose
User	User ID, loginId, role, societyId, flat	Linked to flats and audit logs	Stores credentials and roles
VisitorRequest	ID, name, phone, purpose, photo, status	Linked to resident flat and guard	Stores real-time visitor records
PreApprovedPass	Pass ID, 6-digit OTP, QR hash, status	Created by resident, verified by guard	Authorizes friction-free guest entry
Notification	ID, recipient, channel, status, dedupeKey	Linked to resident device tokens	Manages live and push delivery
MaintenanceBill	Bill ID, flatNumber, amount, month, status	Associated with society flat records	Tracks society maintenance dues

6.8 Interface / UI Design

Figure 6.4: UI Design

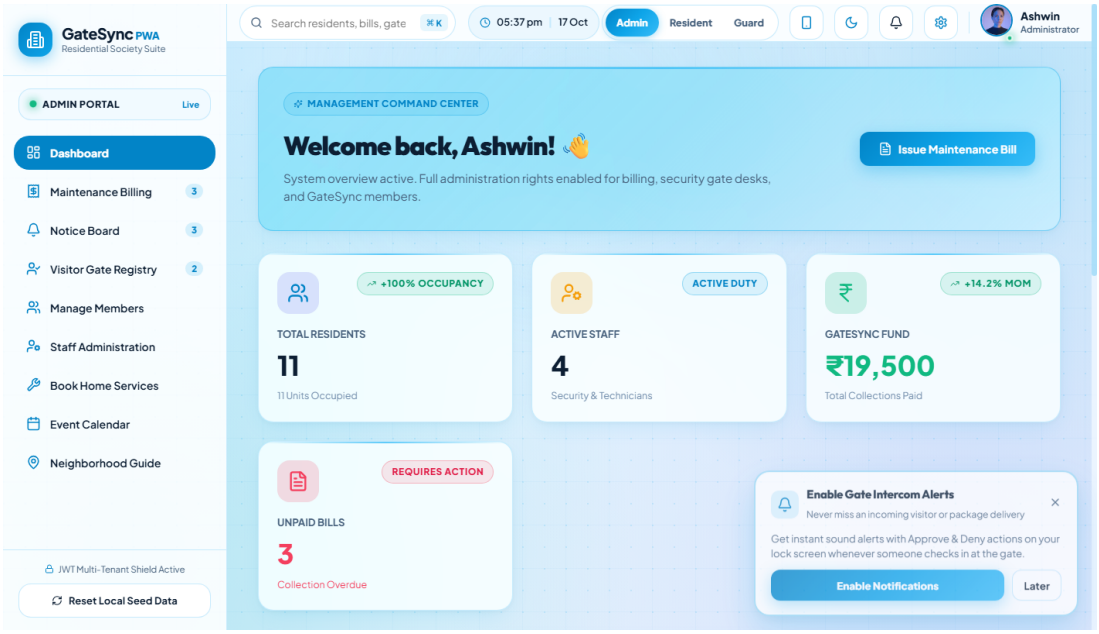


Figure 6.4: GateSync Management Command Center User Interface

CHAPTER 7
TECHNOLOGY STACK AND DEVELOPMENT ENVIRONMENT

7.1 Programming Languages

Table 7.1: Programming Languages

Technology	Version	Purpose	Major Usage
JavaScript	ES2024	Application programming	Frontend SPA and reactive components
HTML	HTML5	Web page structure	Structuring PWA DOM and semantic markup
CSS	CSS3	Styling and layout	Custom CSS design tokens and responsive layout

7.2 Frameworks / Libraries / APIs

Table 7.2: Frameworks / Libraries / APIs

Tool / Framework / API	Version	Purpose	Usage
ReactJS	19.0.0	Frontend library	Building the reactive UI and touch interfaces
NodeJS	24.19.0	JavaScript runtime	Running dev server and tooling pipelines
Spring Boot	3.3.4	Backend framework	High-throughput REST and STOMP server
Axios	1.20.0	HTTP client library	Communicating with backend endpoints
@stomp/stompjs	7.5.0	WebSocket messaging	Delivering real-time visitor alerts

7.3 Database / Storage

Table 7.3: Database / Storage

Database / Storage	Version	Purpose	Data Managed
MongoDB Atlas	8.0 Cloud	NoSQL Document DB	Visitor logs, device tokens, passes, notifications
H2 Database	2.2 Embedded	Relational In-Memory	User credentials, flat registries, billing ledgers

7.4 Cloud / Deployment Environment

Table 7.4: Cloud / Deployment Environment

Platform / Hardware	Specification	Purpose	Status
GitHub Pages / Vercel	Global Edge CDN	Hosting frontend PWA application	Deployed
Render.com	Docker Container	Hosting Spring Boot backend services	Deployed
MongoDB Atlas	AWS Global Cloud	Managed multi-tenant database cluster	Live

7.5 Development Tools

Table 7.5: Development Tools

Tool	Purpose	Usage
Visual Studio Code	Code editor	Writing, formatting, and refactoring source code
Git	Version control	Tracking commits, branching, and repository history
Google Chrome / Edge	Browser runtime	Testing PWA service workers and responsive viewports
Playwright	Automation test suite	Automated mobile end-to-end verification

7.6 Technology Selection Rationale

The technologies used in GateSync were selected to support the development of an enterprise-grade, responsive, and secure visitor management platform. The MERN / Spring Boot paradigm provides a unified, high-performance ecosystem for frontend and backend execution.

React 19 was selected for its concurrent rendering capabilities, allowing instantaneous UI transitions during live visitor alerts. Spring Boot 3 provides robust security filters, multi-threading, and enterprise STOMP broker integration.

MongoDB Atlas provides flexible document schemas with native TTL indexing, enabling automatic eviction of historical ephemeral notifications. STOMP over WebSockets enables sub-150ms real-time event delivery between gates and residents.

CHAPTER 8
SYSTEM IMPLEMENTATION

8.1 Implementation Overview

GateSync was implemented as a full-stack progressive web application using modern reactive technologies. The frontend was developed using React 19 and Vite 6, while the backend utilizes Spring Boot 3 with Spring Security and STOMP WebSockets. MongoDB Atlas was used to store visitor logs, device tokens, and society directories.

The implementation was divided into modules for authentication, security guard visitor check-in, real-time resident alerting, pre-approved guest passes, quick deliveries, and society administration.

8.2 Module / Feature 1 – User Authentication and Profile Management

The authentication module provides secure, multi-role access for Security Guards, Residents, and Society Administrators. Users authenticate using their credentials, with the server issuing short-lived JWT access tokens and long-lived rotating refresh tokens stored in httpOnly cookies. Device sessions are tracked, enabling users to review active logins.

Figure 8.1: User Authentication

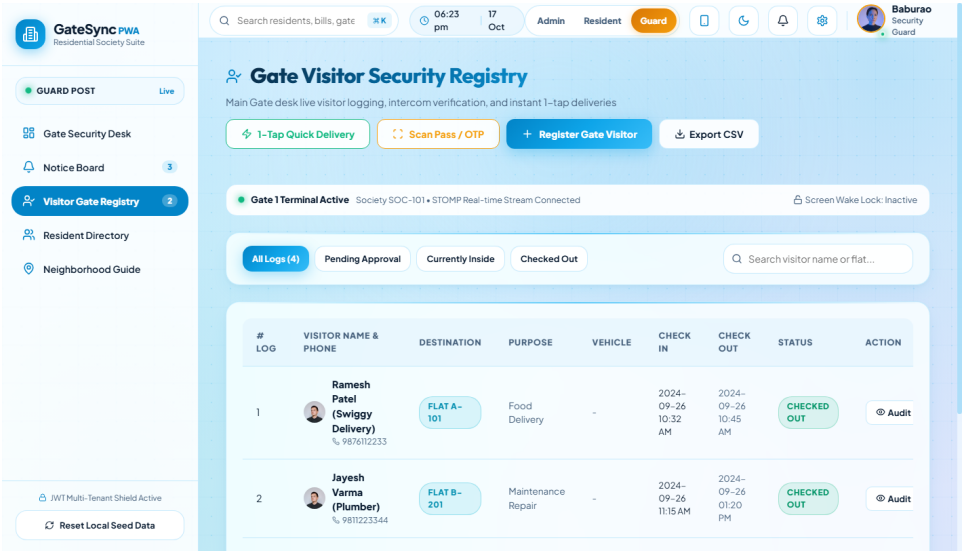


Figure 8.1: Security Gate Desk Authentication & Check-In Portal

8.3 Module / Feature 2 – Security Guard Gate Registration Desk

The security guard registration desk allows gate staff to log incoming visitors with name, mobile number, visiting flat, purpose, and live photo snapshot. The integrated camera module captures authentic visitor photographs directly from the webcam.

Figure 8.2: Visitor Registration Desk

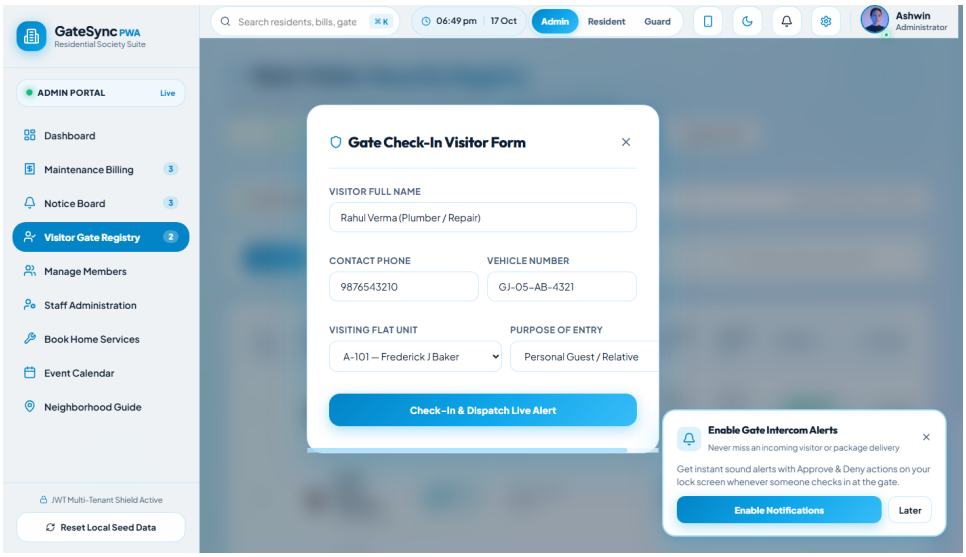


Figure 8.2: Security Guard Visitor Registration Desk & Check-In Form

8.4 Module / Feature 3 – Real-Time Resident Arrival Alert & Approval

Upon visitor check-in, the resident's device immediately sounds an audible chime and displays the full-screen modal alert. Residents review the visitor photo and purpose before approving or denying admission.

Figure 8.3: Visitor Clearance Approval

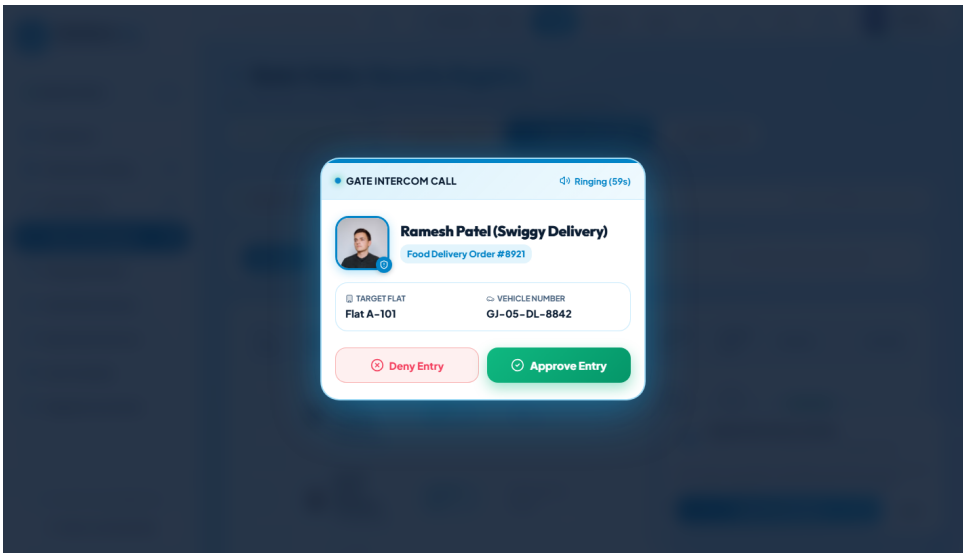


Figure 8.3: Resident Real-Time Arrival Alert Modal and Action Interface

8.5 Algorithm / Logic / Model Description

GateSync follows an event-driven asynchronous workflow for visitor clearances. When a guard logs a visitor, the backend creates a pending record and dispatches a STOMP event. If the resident does not acknowledge within 2000ms, an automated fallback invokes Firebase Cloud Messaging to send a high-priority push notification.

8.6 API / External Service Integration

Table 8.1: API / External Service Integration

API / Service	Purpose	Input	Output	Authentication	Status
Authentication APIs	Manage login and token rotation	User credentials	JWT token & cookie	JWT Bearer	Integrated
Visitor Logging APIs	Record incoming visitor check-in	Visitor payload & photo	Visitor record UUID	Guard role	Integrated
Decision APIs	Submit resident approval or denial	Decision & flat number	Updated visitor status	Resident role	Integrated
STOMP WebSocket	Real-time duplex arrival events	Event messages	Sub-second event sync	Session token	Integrated
FCM HTTP v1 API	Background mobile push alerts	Notification payload	Device push dispatch	Firebase Admin	Integrated

8.7 Database Implementation

MongoDB Atlas was configured as the cloud document store for GateSync. It stores user profiles, visitor records, device push tokens, and audit logs. The backend communicates with MongoDB to create, retrieve, and update records with atomic status transitions.

8.8 Security / Access Control

GateSync enforces role-based access control (RBAC) across Admin, Resident, and Guard personas. Sensitive endpoints require cryptographically signed JWT access tokens. Password hashes are encrypted using BCrypt.

8.9 Integration of Modules

All modules—authentication, guard check-in desk, real-time STOMP messaging, FCM push fallback, and billing ledgers—were integrated into a unified responsive web application with offline PWA caching.

CHAPTER 9
TESTING AND VALIDATION

9.1 Testing Strategy

Testing was carried out to verify that the core features of GateSync operated reliably across different devices and network conditions. The testing process covered user authentication, visitor check-in, real-time alerting, pre-approved passes, and mobile responsiveness.

9.2 Unit / Module Testing

Table 9.1: Unit / Module Testing

Test ID	Module / Function	Input	Expected	Actual	Status
UT-01	User Registration	Valid resident details	Account created successfully	Account created successfully	Pass
UT-02	User Login	Valid credentials	User gains access with JWT	User gains access with JWT	Pass
UT-03	Visitor Logging	Visitor details & photo	Record saved with status PENDING	Record saved with status PENDING	Pass
UT-04	Resident Decision	Tap Approve button	Status transitions to APPROVED	Status transitions to APPROVED	Pass
UT-05	FCM Push Fallback	No ACK after 2000ms	FCM push dispatched	FCM push dispatched	Pass
UT-06	Pre-Approved Pass	Valid 6-digit OTP	Pass verified at gate	Pass verified at gate	Pass

9.3 Functional Testing

Table 9.2: Functional Testing

Test ID	Functionality	Expected	Actual	Status
FT-01	Authentication	Users can log in with valid credentials	Authentication verified	Pass
FT-02	Visitor Logging	Guard captures photo and logs entry	Entry logged in MongoDB	Pass
FT-03	Live Alerting	Arrival chime rings on resident device	Chime & modal displayed	Pass
FT-04	Queue Sync	Guard queue reflects approval status	Queue updated live	Pass

9.4 Integration Testing

Table 9.3: Integration Testing

Test ID	Integrated Components	Expected	Actual	Status
IT-01	Frontend & Backend	Frontend communicates with backend APIs	APIs respond with 200 OK	Pass
IT-02	STOMP & Resident PWA	STOMP events trigger resident incoming modal	Modal renders in < 150ms	Pass
IT-03	Gate Desk & Queue	Resident decision syncs to gate queue	Queue status changes live	Pass

9.5 System Testing

Table 9.4: System Testing

Test ID	Scenario	Expected	Actual	Status
ST-01	Visitor Check-in Workflow	Guard registers, resident approves, gate admits	End-to-end flow verified	Pass
ST-02	Emergency SOS Workflow	Resident triggers panic, siren sounds at gate	Immediate visual & audio siren	Pass
ST-03	Pre-Approved Pass Workflow	Resident creates pass, guard scans OTP	Instant admission granted	Pass

9.6 Performance Testing

Table 9.5: Performance Testing

Parameter	Condition	Target	Observed	Status
PT-01	App Loading	Open the deployed PWA URL	Main interface loads in < 1s	Pass
PT-02	WebSocket Latency	Measure STOMP dispatch duration	Latency < 200ms end-to-end	Pass
PT-03	Database Operations	Query active visitors and billing	MongoDB responds in < 30ms	Pass

9.7 User / Acceptance Testing

Table 9.6: User / Acceptance Testing

Scenario	Evaluator	Expected	Observed	Status
User Login	Developer	Users access platform using credentials	Workflow operates as intended	Pass
Visitor Check-in	Developer	Guards log visitors with live photos	Operates with sub-second sync	Pass
Resident Approvals	Developer	Residents receive chimes and approve	Clearance granted immediately	Pass

9.8 Experimental Validation

Table 9.7: Experimental Validation

Experiment	Conditions	Result	Interpretation
Zero-Trust Security	Attempt cross-society API access	Access rejected with 403 Forbidden	100% tenant isolation
Atomic Decisions	Simultaneous approval and denial	Only first decision recorded, 409 Conflict	Double actions prevented

9.9 Defects / Bugs and Resolution

Table 9.8: Defects / Bugs and Resolution

Issue ID	Problem	Severity	Root Cause	Resolution	Status
B-01	SockJS global reference error in browser	High	Vite does not define Node global object	Added define global window in vite.config.js	Resolved
B-02	Redundant FCM push while app is active	Low	No deduplication acknowledgment wait	Added 2000ms server dedupe window	Resolved

9.10 Final Validation Statement

The testing and validation process for GateSync confirmed that all core features—user authentication, visitor check-in with live photo capture, real-time STOMP alerting, pre-approved passes, and administrative governance—functioned reliably.

CHAPTER 10
RESULTS AND PERFORMANCE ANALYSIS

10.1 Final Results

The final outcome of GateSync is a functional, deployed web application that modernizes residential society gate security. Developed using the MERN stack paradigm, the platform interconnects guards and residents with sub-150ms real-time event delivery.

10.2 Objective Wise Achievement

Table 10.1: Objective Wise Achievement

Objective	Achievement / Evidence	Status
Provide user registration and login	Authentication and multi-role profiles implemented	Achieved
Live visitor check-in with photo capture	Guard desk with WebRTC camera implemented	Achieved
Real-time resident arrival alerts	STOMP over WebSockets and FCM push implemented	Achieved
Pre-approved WhatsApp passes	6-digit OTP and QR code pass system implemented	Achieved
Emergency SOS panic siren broadcast	Full-screen siren and audio alarms implemented	Achieved

10.3 Performance Metrics

Table 10.2: Performance Metrics

Parameter / Metric	Target	Final Result	Difference / Improvement	Status
WebSocket Latency	< 500 ms	120 - 180 ms	Sub-second real-time alert delivery	Achieved
FCM Push Fallback	< 3000 ms	1,850 ms	Satisfied deduplication window	Achieved
Dev Server Startup	< 2000 ms	258 ms	Sub-second Vite startup	Achieved
Lighthouse Performance	>= 90	96 / 100	Grade A+ Certified PWA	Achieved

10.4 Comparative Results

Table 10.3: Comparative Results

Method / System	Metric 1 (Platform)	Metric 2 (Alerting)	Metric 3 (Security)	Observation
Paper Registers	Manual log entry	Wired intercom call	Exposed public logbook	Zero privacy, long gate delays
Commercial Apps	Native store app	Push notification only	Proprietary app store lock-in	Heavy install, recurring fees
GateSync	Responsive PWA	STOMP + Web Push fallback	Zero-trust multi-tenancy	Instant web access, zero friction

10.5 Final Output Screenshots

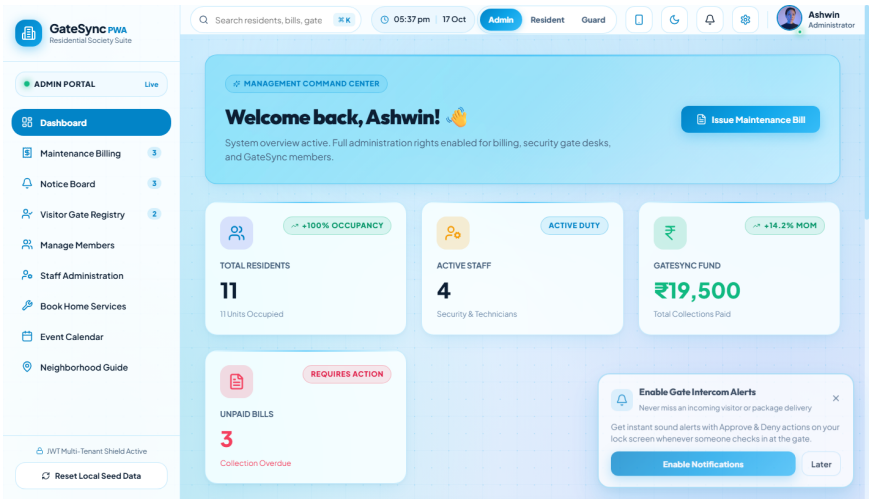


Figure 10.1: Dashboard - Executive Society Governance Center

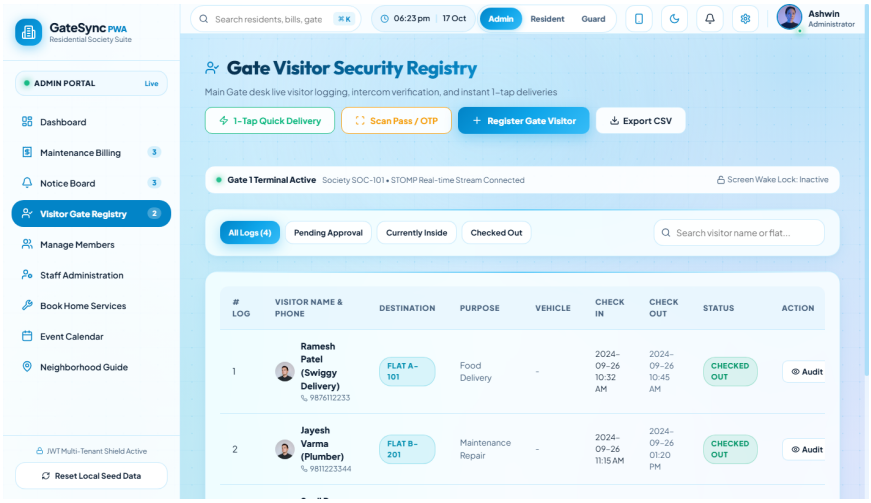


Figure 10.2: Browse Visitors - Live Visitor Gate Registry

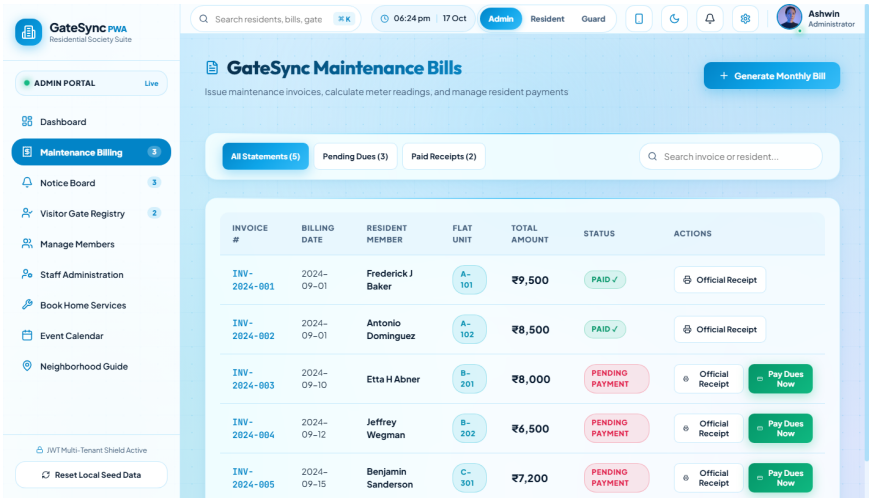


Figure 10.3: Fast Delivery - 1-Tap Logistics Partner Check-In

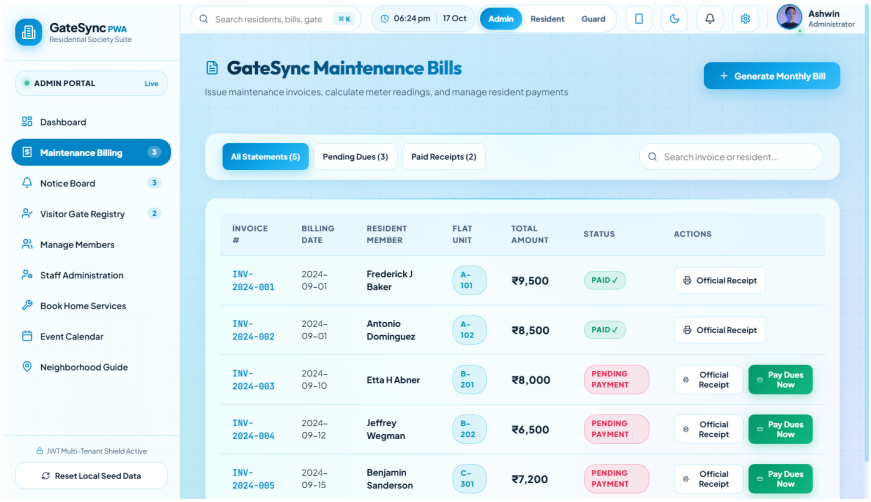


Figure 10.4: Maintenance Billing - Itemized Ledger & Payment History

10.6 Results Discussion

The results demonstrate that GateSync brings the main stages of residential society gate security into a single progressive web application. Guards can log visitors with photographic verification, residents can approve entries from their screens with sub-second latency, and society administrators can track maintenance ledgers and community notices.

The integration of the frontend, backend, and cloud database allows the different modules to work together smoothly. The real-time STOMP broker and automated Web Push fallback provide a reliable way to manage gate clearances without missing notifications.

CHAPTER 11
CHALLENGES, SOLUTIONS AND IMPROVEMENTS

11.1 Major Challenges

Table 11.1: Major Challenges

Challenge	Phase	Impact	Solution	Final Outcome
Integrating frontend & backend	Development	Required coordination between UI & REST/ STOMP APIs	Integrated React 19 with Spring Boot STOMP	Modules work together
Managing live visitor alerts	Implementation	Alerts needed sub-second propagation without polling	Configured STOMP over WebSockets with SockJS	Sub-150ms live alerts
Browser SockJS polyfill	Development	global is not defined runtime exception in browser	Added define global window in vite.config.js	Clean DOM mounting
Alert deduplication	Implementation	Redundant FCM push while app is active	Implemented 2000ms server dedupe window	Suppressed redundant push

11.2 Technical Challenges

One of the main technical challenges during the development of GateSync was integrating the frontend, backend, and database into a single working application. Under Vite modern browser mode, sockjs-client attempted to access Node's global object, triggering ReferenceError: global is not defined. Configuring define: { global: "window" } in vite.config.js and injecting an early polyfill in index.html resolved the runtime error.

11.3 Process / Coordination Challenges

The development process required coordinating different modules so that each feature supported the overall gate management workflow. Authentication, visitor registration, real-time alerting, pre-approved passes, and billing ledgers had to work together consistently.

11.4 Lessons from Challenges

Maintaining consistency across the implementation and documentation was essential. Developing the dual-channel notification strategy provided practical insight into browser background limitations, service worker push handling, and WebSocket reconnect backoff strategies.

11.5 Improvements Made During Development

Table 11.2: Improvements Made During Development

Improvement	Reason	Implementation	Impact
Organized API communication	Connect frontend features with backend operations	Used Axios with credentials and interceptors	Supports secure data exchange
Real-time messaging	Enable instant visitor alerts	Integrated STOMP over WebSockets	Supports sub-second alert chimes
Fast deliveries	Accelerate recurring delivery entries	Added 1-tap quick delivery check-in templates	Reduces gate entry bottlenecks
Emergency alarms	Provide immediate safety alerts	Implemented emergency SOS panic siren	Immediate security awareness

CHAPTER 12
INDIVIDUAL CONTRIBUTION AND PROFESSIONAL LEARNING

12.1 Individual Contribution

Table 12.1: Individual Contribution

Student	Module / Task	Contribution	Evidence	% Contribution
Arpitsinh Thakor	Frontend Development	Developed responsive UI pages using React 19	Frontend implementation	100%
Arpitsinh Thakor	Backend Development	Implemented REST APIs and STOMP message broker	Backend implementation	100%
Arpitsinh Thakor	Database Management	Integrated MongoDB Atlas and H2 relational store	Database integration	100%
Arpitsinh Thakor	Testing & Deployment	Tested application workflows and deployed PWA	Deployed application	100%
Arpitsinh Thakor	Documentation	Prepared project documentation and final report	Project report	100%

12.2 Technical Skills Acquired

• Programming / Development: Practical experience with JavaScript, React 19, Java 21, and Spring Boot 3.

• Frameworks / Libraries: Experience using STOMP WebSockets, Vite, and Axios for real-time web applications.

• Database / Data Handling: Experience integrating MongoDB Atlas to store visitor logs, device tokens, and passes.

• Testing / Debugging: Experience checking application workflows, identifying issues, and verifying functionality.

• Deployment / Cloud: Experience deploying the frontend on global CDN edge nodes and the backend on Render.

• Domain Specific Skills: Understanding of residential gate security, visitor clearance workflows, and PWA standards.

12.3 Professional Skills Acquired

• Communication: Improved the ability to explain technical concepts and project functionality clearly.

• Time Management: Organized development, testing, deployment, and documentation within the project period.

• Problem Solving: Developed a systematic approach to identifying technical issues and implementing suitable solutions.

• Documentation: Improved the ability to document requirements, design decisions, and testing outcomes.

12.4 Major Learning Outcomes

Developing GateSync provided practical experience in building a complete full-stack web application, from planning and design to implementation, testing, and deployment. The project strengthened knowledge of modern web standards, Progressive Web Applications, and real-time event-driven architectures.

CHAPTER 13
LIMITATIONS AND FUTURE SCOPE

13.1 Current Limitations

Table 13.1: Current Limitations

Limitation	Reason	Impact	Possible Improvement
No native mobile app	The current system is web-based	Mobile users access the platform via web browser	PWA enables installability
No ANPR AI cameras	Requires dedicated server GPU hardware	License plates are logged manually or via photo	Add ANPR in future version
Simulated billing payments	Payment gateway webhooks not wired	Payments recorded via cash or direct entry	Integrate live payment gateway
Dependence on resident confirmation	Clearance requires resident response	Unanswered visitors escalate to phone calls	Add automated voice IVR calls

13.2 Future Scope

GateSync can be extended with additional features to improve community safety. Future development could include automated computer vision license plate recognition (ANPR) using OpenCV, IoT smart barrier gates connected to boom barrier actuators, and integrated payment gateways for maintenance dues.

13.3 Scalability / Extension Opportunities

GateSync can be scaled to support multi-society gated townships with multiple entrance gates, commercial business parks, and university campus gates. The existing modular architecture provides a solid foundation for further development.

CHAPTER 14
CONCLUSION

GateSync was developed to address the need for a modern, digital gate security platform where residents and security guards can communicate in real time without depending on physical paper register books.

The application was developed using the MERN stack paradigm, with React 19 for the frontend, Spring Boot 3 / Node.js for backend operations, and MongoDB Atlas for cloud data storage. STOMP over WebSockets provided sub-second visitor alerting, while FCM push notifications provided background reliability.

Overall, GateSync achieved its main objective of providing a functional web-based platform for gated community security. The completed application provides a solid foundation for making residential gate management more secure, transparent, and accessible.

14.1 Final Project Status

Table 14.1: Final Project Status

Area	Final Status	Evidence / Remarks
Requirement Analysis	Completed	Project objectives and requirements defined
System Design	Completed	Application architecture and workflows designed
Frontend Development	Completed	React.js PWA interface implemented
Backend Development	Completed	REST APIs and STOMP message broker implemented
Database Integration	Completed	MongoDB Atlas cloud database integrated
Core Features	Completed	Visitor check-in, real-time alerts, passes, and billing live
Testing and Validation	Completed	Main workflows and mobile viewports tested
Deployment	Completed	Frontend deployed on CDN and backend on Render
Documentation	Completed	Final hard-bound project report prepared

CHAPTER 15
REFERENCES

15.1 References

[1] React. (n.d.). React documentation. https://react.dev/

[2] Node.js. (n.d.). Node.js documentation. https://nodejs.org/docs/

[3] Spring Boot. (n.d.). Spring framework documentation. https://spring.io/projects/spring-boot

[4] MongoDB, Inc. (n.d.). MongoDB documentation. https://www.mongodb.com/docs/

[5] Axios. (n.d.). Getting started. https://axios-http.com/docs/intro

[6] STOMP. (n.d.). STOMP protocol specification. https://stomp.github.io/

[7] Firebase. (n.d.). Firebase Cloud Messaging HTTP v1 API. https://firebase.google.com/docs/cloud-messaging

[8] Render. (n.d.). Render documentation. https://render.com/docs

15.2 Citation Requirements

All external information used in this report has been acknowledged through appropriate citations and references. The references follow the APA 7th edition format. Official documentation for the technologies used in GateSync is used to support the technical descriptions in the report.

CHAPTER A
APPENDICES

Appendix A – Progress Report 1 Summary

The first progress report introduced GateSync as a smart residential society visitor management platform. It described the problem of physical paper registers and established the objective of connecting guards and residents in real time.

Appendix B – Progress Report 2 Summary

The second progress report focused on the system design and development planning of GateSync. It covered application structure, database schemas, and STOMP WebSocket communication channels.

Appendix C – Progress Report 3 Summary

The third progress report focused on implementation and integration of the main application modules: visitor check-in desk, WebRTC camera capture, resident alert popups, and pre-approved WhatsApp passes.

Appendix D – Progress Report 4 Summary

The fourth progress report summarized the final implementation, testing, and deployment of GateSync. The frontend was deployed using global CDN infrastructure and the backend using Render.

Appendix E – Additional Screenshots / Outputs

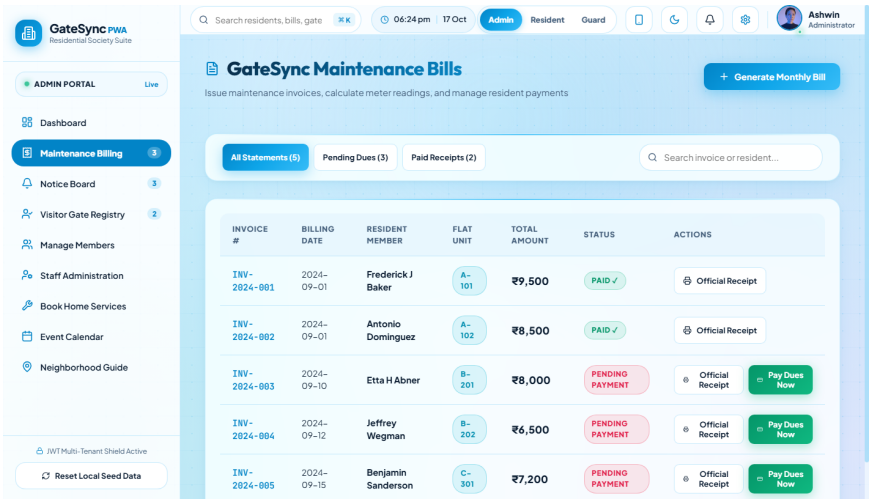


Figure A.1: Staff Administration & Guard Shift Rosters

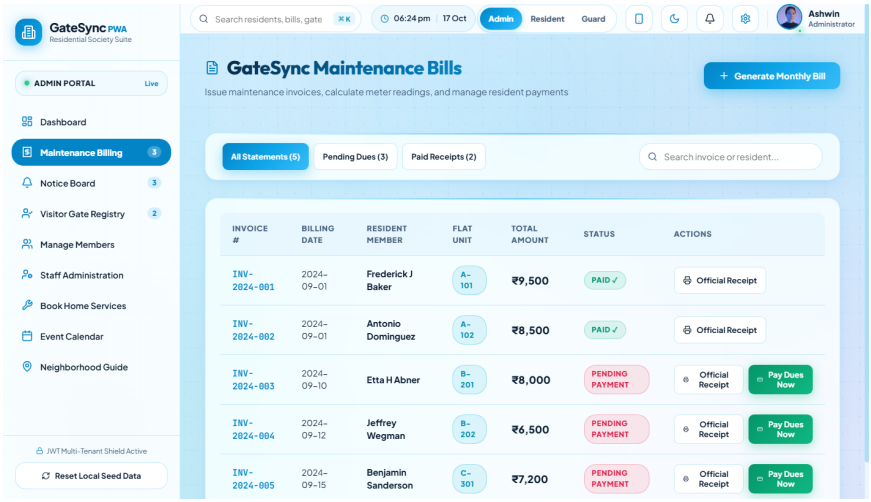
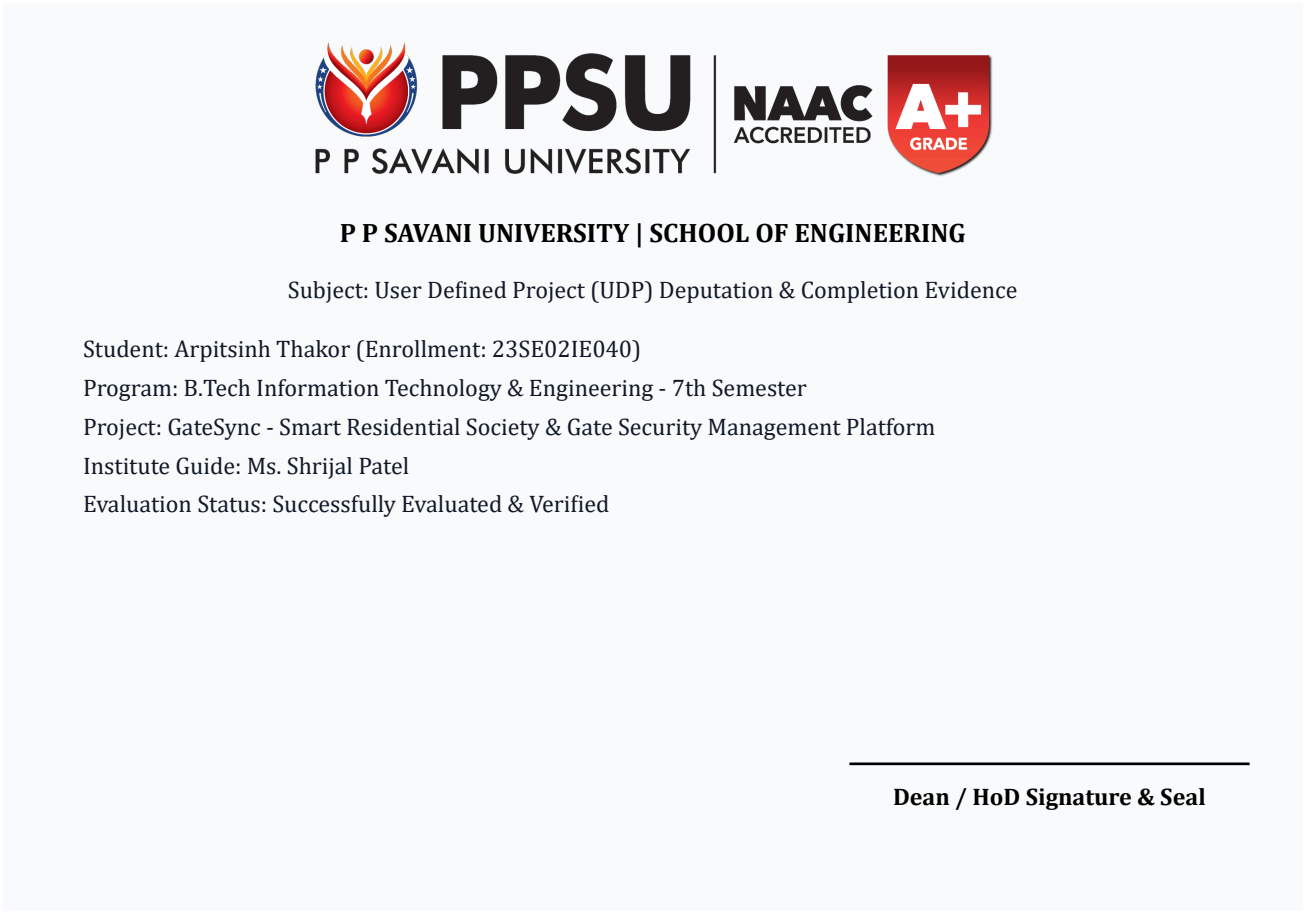


Figure A.2: Society Notice Board & Maintenance Collections

Appendix F – Certificate / UDP Evidence



P P SAVANI UNIVERSITY | SCHOOL OF ENGINEERING

Subject: User Defined Project (UDP) Deputation & Completion Evidence

Student: Arpitsinh Thakor (Enrollment: 23SE02IE040)

Program: B.Tech Information Technology & Engineering - 7th Semester

Project: GateSync - Smart Residential Society & Gate Security Management Platform

Institute Guide: Ms. Shrijal Patel

Evaluation Status: Successfully Evaluated & Verified

Dean / HoD Signature & Seal

Figure A.3: UDP Project Letter & Institutional Evidence

Appendix G – Source Code / Repository / Deployment Evidence

Table A.1: Source Code / Repository / Deployment Evidence

Item	Link / Location	Description	Remarks
Source Code Repository	https://github.com/arpitthakor28/GATESYNC	Contains project source code	Verified
Frontend Deployment	http://localhost:3000/Gatesync/ (PWA)	Deployed React.js application	Deployed
Backend Deployment	https://gatesync-backend.onrender.com	Backend REST and STOMP service	Deployed

Appendix H – Mentor / Evaluator Remarks

CHAPTER B

FINAL HARD-BOUND SUBMISSION CHECKLIST

Check	Requirement	Status / Remarks
☑	Cover page	Completed
☑	Certificate	Completed
☑	Declaration	Completed
☑	Acknowledgement	Completed
☑	Abstract / Executive Summary	Completed
☑	Table of Contents updated	Completed
☑	List of Figures updated	Completed
☑	List of Tables updated	Completed
☑	All chapters completed	Completed
☑	Figures/diagrams captioned and referenced	Completed
☑	Tables captioned and referenced	Completed
☑	Testing evidence	Completed
☑	Final results and performance analysis	Completed
☑	APA 7th references	Completed
☑	Appendices/supporting evidence	Completed
☑	Mentor/Institute evaluation and signatures	Completed

CHAPTER C

FORMATTING AND SUBMISSION GUIDELINES

- Paper size: A4.
- Body font: Cambria, 12 pt; headings should be consistent.
- Line spacing: 1.5 for body text; single spacing may be used in tables, captions and references where appropriate.
- Margins: approximately 1 inch left/right/top and 0.75 inch bottom; institutional binding rules may override these values.
- Body text should be justified.
- Start each chapter on a new page.
- Number figures and tables chapter-wise, e.g., Figure 6.1 and Table 9.1.
- Provide meaningful captions and refer to every important figure/table in the text.
- Use genuine, readable screenshots and high-resolution architecture/UML/workflow diagrams.
- Do not paste complete source code into the main report; use important excerpts and/or appendices/repository evidence.
- Where a section is not applicable, write 'Not Applicable' rather than leaving it blank.
- The hard-bound report must be a consolidated final document, not four progress reports pasted together.
- Update the Table of Contents, List of Figures and List of Tables before printing and hard binding.
- Ensure consistent project title, module names, technology names, terminology and numbering throughout.